

Name: Iman Haamid

Course: ITAI 2376

Final Reflection



Project Reflection & Technical Updates



Evolution from Blueprint

My final implementation for the **Personal Time Management and Budget Agent** successfully expanded upon the original **Option B (Multi-Agent System)** architecture. While the core mission remained the automation of my financial ecosystem across Interactive Brokers, Fidelity, and Ally, several key technical enhancements were made during the build phase:

- **Multimodal Interface (New):** Originally planned as a standard script, the final project features a **Gradio-powered** interface that supports both text and real-time voice input via microphone. (next version will have the microphone)
- **Voice Briefing Logic (New):** I integrated a Text-to-Speech (TTS) engine using **gTTS** to allow the "Steward" to provide verbal financial recommendations, enhancing the "Financial Chief of Staff" experience.
- **Refined Agent Brain:** The orchestration was optimized using **Gemini 3.1 Flash-lite** to serve as the high-speed "brain" between the Accountant and Scheduler agents.



Technical Accomplishments

- **LSTM Cash-Flow Forecasting:** Successfully implemented a Long Short-Term Memory (LSTM) network to analyze historical spending patterns in categories like groceries and gas to predict end-of-month liquidity.
- **Agentic Role-Based Orchestration:** Utilized the **CrewAI** framework to maintain clear role separation. The "Accountant" handles the verified budget data (\$5,626.19 surplus) while the "Scheduler" maps these findings to 30-day calendar roadmaps.
- **Security & Best Practices:** Adhered to the planned data privacy measures by using **environment variables** and **.gitignore** protocols to protect sensitive API keys.

Technical Implementation: Agent vs. Model Logic

In the final build, I made a strategic decision to utilize a **Multi-Agent Orchestration (CrewAI)** rather than a simple chatbot model. This was critical for the following reasons:

- **Data Grounding:** While the **Gemini 3.1 Flash** model provides the reasoning, the **Agents** act as the functional "hands" that utilize specialized tools (RAG and CSV Readers) to fetch real-time financial data.
- **Conflict Resolution:** The agents were programmed with **"Hard Stop" logic**. In the final demo, when queried about a decreasing surplus, the agent correctly prioritized

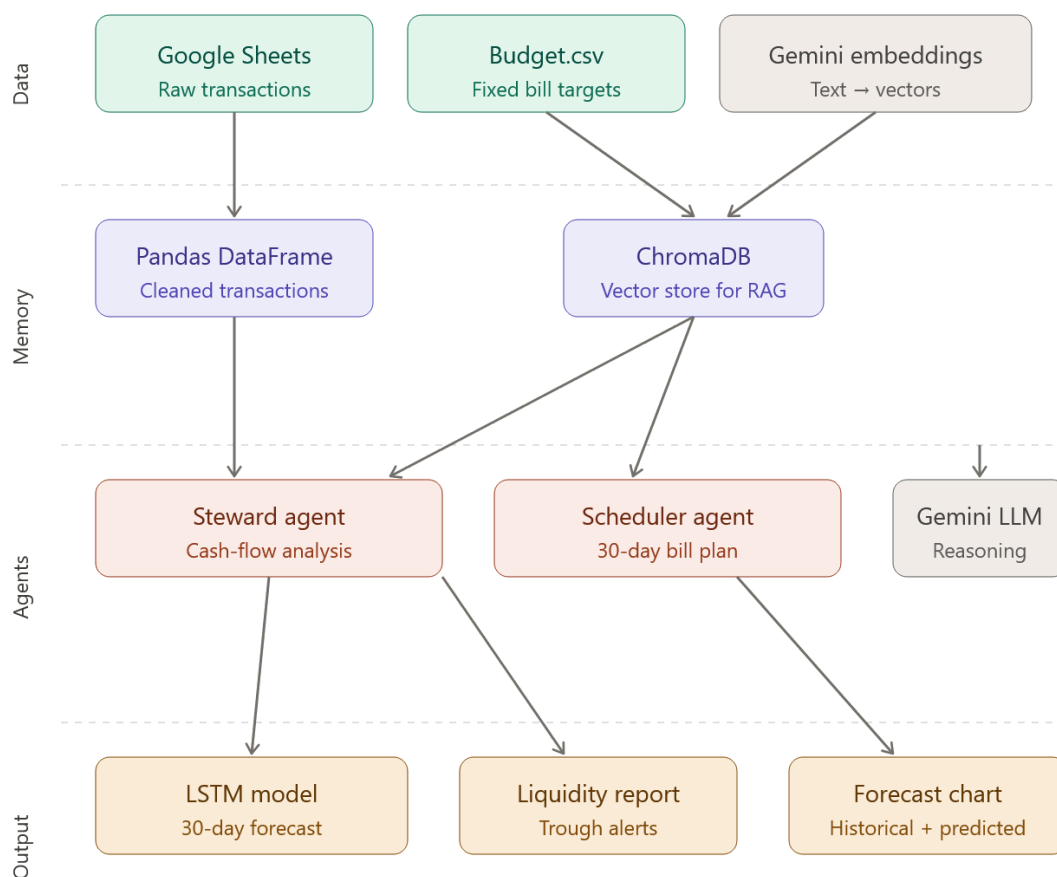
mortgage stability over discretionary spending, a level of nuanced reasoning that standard LLMs often struggle with without agentic guardrails.

- **Voice Integration:** The transition to a **Gradio 6.0** interface allowed for a multimodal "Talking Steward" experience, proving that complex financial backends can be made accessible through conversational AI.

Lessons Learned

The primary challenge encountered was managing the **cognitive load** of multi-step reasoning. By implementing structured Markdown outputs and voice-to-voice loops, I was able to bridge the gap between static budgeting and real-time financial intelligence.

Updated Diagram:



Midterm Blueprint

Personal Time Management and Budget Agent which is a highly practical alternative, especially since I already manually manage a detailed monthly budget for bills, groceries, and gas. This project will allow me to automate the financial consolidation strategy I am currently executing across **Interactive Brokers, Fidelity, and Ally**.

1. Problem Statement

The Problem: Managing a complex financial ecosystem—including personal trading accounts, business accounts (Tastytrade), and children's 529 plans—requires significant manual effort to track and optimize. This process is currently plagued by fragmented data across various institutions, leading to potential inefficiencies in cash-flow management and missed opportunities for strategic rebalancing. Without an automated solution, the user must manually reconcile balances from Interactive Brokers, Fidelity, and Ally to maintain a clear picture of their net worth and monthly liquidity.

Beyond simple tracking, the manual nature of these tasks creates a cognitive load that can interfere with high-stakes trading decisions. By automating the data aggregation and categorization of recurring expenses like groceries and gas, the user can shift their focus from administrative maintenance to active portfolio management. This project seeks to bridge the gap between static monthly budgeting and real-time financial intelligence.

The Benefit: Functioning as a "**Financial Chief of Staff**," this agent will automate expense categorization, track monthly income withdrawal thresholds, and recommend dedicated time-slots for deep-focus trading.

2. Option Choice: Option B (Multi-Agent System)

I am choosing **Option B**. This problem is best solved by specialized roles collaborating:

- **Agent 1 (The Accountant):** Focuses on budget tracking and financial consolidation.
- **Agent 2 (The Scheduler):** Focuses on time-blocking and productivity. Explaining this choice: A multi-agent system allows the "Accountant" to handle sensitive financial data while the "Scheduler" manages the calendar, preventing a single reasoning loop from becoming overwhelmed by unrelated tasks.

3. Agent Architecture

- **[INPUT]:** A user request (e.g., "Can I afford to withdraw \$1,000 from IBKR this month?").
- **[DEEP LEARNING MODEL]:** Recurrent Neural Network (RNN) / Long Short-Term

Memory (LSTM) for cash-flow forecasting or a **Bidirectional Encoder Representations from Transformers (BERT) Classifier** to route the request to the correct agent.

- **[AGENT BRAIN]: LLM (Gemini 3.1 Flash lite)** acting as the orchestrator between sub-agents.
- **[TOOLS]: Google Sheets API** (for the budget), **Google Calendar API**, and a **Banking/Brokerage API** (read-only).
- **[OUTPUT]:** A consolidated financial update and a suggested calendar block for the withdrawal task.

4. Deep Learning Connection

- **Transformers (Module 05):** I will use a **BERT-based model** to classify and extract entities from financial statements and calendar invites, ensuring the agent understands the difference between a "bill" and a "subscription".
- **RNNs/LSTMs (Module 06):** I will implement a Long **Short-Term Memory (LSTM) network** to analyze your historical spending patterns and predict your end-of-month balance based on current gas and grocery trends.

5. Agent Framework: CrewAI

I plan to use **CrewAI** because it is specifically designed for **Role-Based Orchestration and Task Delegation** in multi-agent systems. It allows me to define "Job Titles" (e.g., Budget Analyst) and "Goals" for each agent, which fits the collaborative nature of this project.

6. Tools & Data

- **Tools:** (1) **Financial API** (e.g., Interactive Brokers Client Portal API or Plaid) to securely ingest read-only brokerage and bank data for real-time balance tracking. (2) **Google Calendar API** to programmatically identify focus-time gaps and insert suggested schedule blocks for financial rebalancing. (3) **Google Sheets API** to act as a persistent database for the monthly budget, replacing the manual entries currently used to track groceries, gas, and utilities.
-

Data: To maintain a high level of real-world financial complexity during testing, I will utilize my personal trading logs and anonymized budget information as the primary data

source.

- Trading Data: This includes active scalping logs from XSP and MES trades, captured across platforms like Interactive Brokers and Tastytrade, to simulate high-frequency financial activity.
- Budgetary Data: Categorized historical spending from personal spreadsheets—spanning categories like groceries, gas, and utilities—will provide the necessary variance for the LSTM forecasting model.

7. Build Plan

- **Week 1 (Apr 6–13):** Initialize the **CrewAI** environment and connect the Google Calendar API.
- **Week 2 (Apr 14–20):** Build the **LSTM forecasting model** using historical spending data.
- **Week 3 (Apr 21–27):** Program the agent hand-off logic (Accountant → Scheduler) and test with sample withdrawal queries.
- **Week 4 (Apr 28–May 3):** Record a demo showing the agent managing a budget "crisis" and finalizing the project write-up.

8. Expected Challenges

- **Challenge 1 (Data Privacy):** Financial data is sensitive. I will mitigate this by using **environment variables** for API keys and ensuring no actual account numbers are hard-coded into the script.
- **Challenge 2 (Task Drift):** Agents may lose focus during long multi-step plans. I will use **structured outputs** (JSON) to ensure the agents pass data to each other in a consistent format.

SOURCES & REFERENCES

1. Houston Community College. (2026, Spring). *ITAI 2376 Courseware: Module 06: Sequence Modeling with RNNs and Module 10: The Rise of AI Agents*.
2. BCG AI Platforms Group. (2025, November). *Building Effective Enterprise Agents Briefing*.
3. CrewAI Framework. (2026, April). *Official technical documentation and repository*. GitHub. Retrieved from [<https://docs.crewai.com>].
4. The Complete Guide to AI Agents for Developers: LangChain, CrewAI, and Beyond. (2026, April). *daily.dev*. Retrieved from [<https://daily.dev/blog/complete-guide-ai-agents-developers-langchain-crewai>].

5. CME Group. *Formal contract specifications for the Micro E-mini S&P 500 (MES)*.
6. Brownlee, J. (2018). *Deep Learning for Time Series Forecasting*.
7. Haamid, I. (2026, April). *Gemini Prompt: "is CrewAI the best choice?"* [Personal research note].